

《数据结构与算法设计》

课程设计总结

学号： 2353233

姓名： 程序

专业： 计算机科学与技术

2025 年 7 月

目 录

第一部分 算法实现设计说明.....	3
1.1 题目.....	
1.2 软件功能.....	
1.3 设计思想.....	
1.4 逻辑结构与物理结构.....	
1.5 开发平台.....	
1.6 系统的运行结果分析说明.....	
1.7 操作说明.....	
第二部分 综合应用设计说明.....	18
2.1 题目.....	
2.2 软件功能.....	
2.3 设计思想.....	
2.4 逻辑结构与物理结构.....	
2.5 开发平台.....	
2.6 系统的运行结果分析说明.....	
2.7 操作说明.....	
第三部分 实践总结.....	42
3.1. 所做的工作.....	
3.2. 总结与收获.....	
第四部分 参考文献.....	43

第一部分 算法实现设计说明

1.1 题目（第3题）

给定一个有向图，完成：

- （1）建立并显示它的邻接链表；
- （2）对该图进行拓扑排序，显示拓扑排序的结果，并随时显示入度域的变化情况
- （3）给出它的关键路径（要求显示出 $V_e, V_l, E, L, L-E$ 的结果）

1.2 软件功能

功能 1：提供用户友好的输入方式，支持通过顶点和边的信息构建有向图，并在支持重复边检测与输入合法性验证的情况下根据输入数据动态构建邻接链表结构，同时确保以清晰、可读的格式向用户展示所构建的邻接链表。

实现方式：通过设计合理的输入数据和数据结构、编写专门的输入解析和图形显示函数使得软件能成功且便捷地构建邻接链表，检测合法性并且能通过一系列格式变换清晰地反馈给用户有向图所构建成的邻接链表

功能 2：基于邻接链表结构，实现对图中顶点的拓扑排序，并在排序过程中实时显示各顶点的入度变化情况，帮助用户理解排序过程，之后再输出拓扑排序的最终序列。

实现方式：使用某种数据结构作为辅助队列。在算法主循环中插入打印语句，循环输出当前的中间状态或最终状态。

功能 3：根据拓扑排序结果，先计算每个事件的最早发生时间(V_e)和最迟发生时间(V_l)，再计算每条边（活动）的最早开始时间（E）、最迟开始时间（L）及时间余量（ $L - E$ ），最后再输出所有关键活动并标识出图中的关键路径。

实现方式：在获得拓扑序列之后。定义 $V_e[]$ 和 $V_l[]$ 数组，进行正、反两次遍历计算。最后遍历所有边，计算并输出各项参数。

1.3 设计思想

功能 1：建立并显示它的邻接链表

实现思路：通过实现以下功能来实现：

- (1) 创建邻接链表：通过设计数组+链表的混合数据结构设计来成功创建邻接链表。
- (2) 添加边到链表：需要设计多种插入策略（头插/尾插/有序插入），同时需要有重复边检测和输入验证机制。
- (3) 显示邻接链表：需要做到用户友好式的可视化展示。

拟定算法设计流程如下：

- (1) 初始化顶点数组，并为每个顶点初始化一个空链表。
- (2) 接收用户输入的边信息（包括起点、终点、权重），进行重复边与合法性校验。
- (3) 将合法的边以“头插法”或“尾插法”插入对应顶点的链表中。
- (4) 遍历顶点数组及其链表，以表格或分级列表形式输出邻接链表。

功能 2：对该图进行拓扑排序，显示拓扑排序的结果，并随时显示入度域的变化情况

实现思路：采用 Kahn 算法进行拓扑排序，通过维护一个队列存放当前入度为 0 的顶点，并动态更新相邻顶点的入度。详细步骤如下：

- (1) 计算所有顶点的入度
- (2) 将入度为 0 的顶点加入队列
- (3) 逐步移除入度为 0 的顶点，并更新相邻顶点的入度
- (4) 重复直到所有顶点都被处理

拟定算法设计流程如下：

- (1) 初始化一个入度数组，遍历邻接链表统计每个顶点的入度。
- (2) 将所有入度为 0 的顶点加入队列。
- (3) 从队列中取出顶点并输出，遍历其所有邻接顶点，将其入度减 1。若减后入度为 0，则将该顶点加入队列。
- (4) 在每一步更新入度时，显示当前各顶点的入度值。
- (5) 重复上述过程直至队列为空。若输出的顶点数少于图中顶点总数，则说明图中存在环。

功能 3：给出它的关键路径（要求显示出 $V_e, V_l, E, L, L-E$ 的结果）

我们使用 CPM 算法计算每个物理量：

事件的最早发生时间 V_e ：

初始化: $Ve[0] = 0$ (起始事件)

递推公式: $Ve[j] = \max\{Ve[i] + \text{weight}(i, j)\}$ (对所有指向 j 的边 (i, j))

事件的最迟发生时间 Vl :

初始化: $Vl[n-1] = Ve[n-1]$ (终止事件)

递推公式: $Vl[i] = \min\{Vl[j] - \text{weight}(i, j)\}$ (对所有从 i 出发的边 (i, j))

活动的最早开始时间 $E(i, j): E(i, j) = Ve[i]$

活动的最迟开始时间 $L(i, j): L(i, j) = Vl[j] - \text{weight}(i, j)$

活动的时间余量(松弛时间) $L-E: L-E = L(i, j) - E(i, j) = Vl[j] - Ve[i] - \text{weight}(i, j)$

实现思路即为依次计算每个值的数据并根据操作实时改变数值

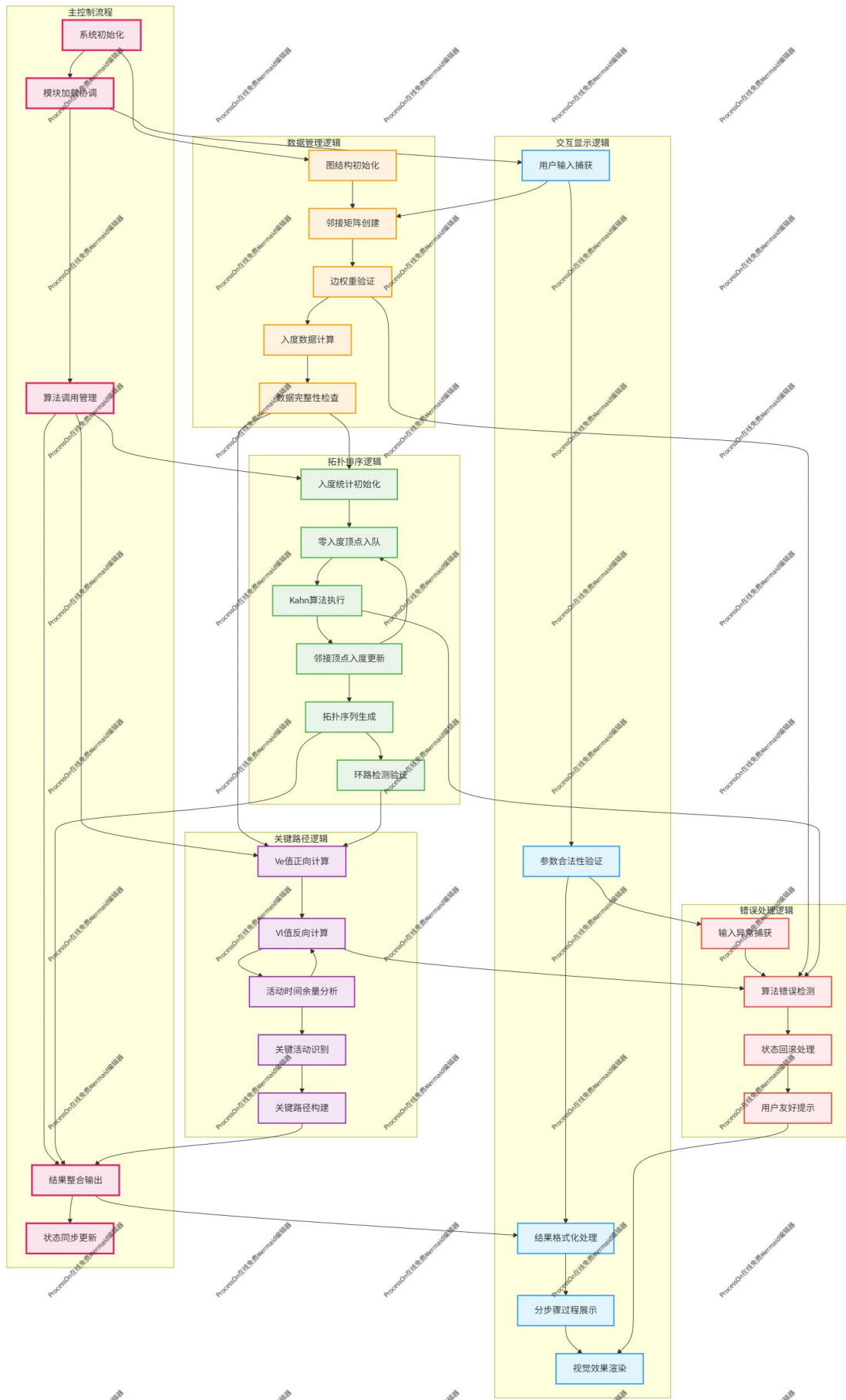
拟定算法设计流程如下:

- (1) 对图进行拓扑排序, 得到顶点序列;
- (2) 正向遍历拓扑序列, 递推计算每个顶点的 Ve ;
- (3) 逆向遍历拓扑序列, 递推计算每个顶点的 Vl ;
- (4) 遍历每一条边, 计算: 活动最早开始时间 $E(i, j)$, 活动最迟开始时间 $L(i, j)$, 时间余量 $L-E$
- (5) 输出所有时间余量为 0 的活动, 即关键活动, 它们构成关键路径。

1.4 逻辑结构与物理结构

1.4.1 逻辑结构

有向图分析系统采用分层架构设计, 从逻辑上分为数据层、算法层、控制层和展示层四个层次。



(1) 数据层 (Data Layer)

核心数据结构:

顶点数据结构:

```
{
  index: 0,           // 顶点编号 (0 到 n-1)
  inDegree: 0,        // 当前入度值
  ve: 0,              // 最早发生时间
  vl: 0,              // 最迟发生时间
  id: "vertex_唯一标识" // 顶点唯一标识符
}
```

边数据结构:

```
{
  from: 0,            // 起始顶点编号
  to: 1,              // 终点顶点编号
  weight: 1,          // 边的权重值
  E: 0,              // 最早开始时间
  L: 0,              // 最迟开始时间
  slack: 0,           // 松弛时间 (L-E)
  isCritical: false,  // 是否为关键边
  id: "edge_唯一标识" // 边唯一标识符
}
```

图数据结构:

```
{
  adjMatrix: [],      // 二维邻接矩阵
  vertexCount: 0,     // 顶点总数
  edgeCount: 0,       // 边总数
  vertices: [],       // 顶点信息数组
  edges: [],          // 边信息数组
  id: "graph_唯一标识" // 图唯一标识符
}
```

数据管理架构 GraphDataManager:

```
└── vertices: MyVector<VertexObject> // 顶点数据容器
└── edges: MyVector<EdgeObject>      // 边数据容器
└── adjMatrix: number[][]           // 邻接矩阵存储
└── inDegree: MyVector<number>      // 入度数据容器
└── ve: MyVector<number>            // 最早时间容器
```

```

├── v1: MyVector<number>           // 最迟时间容器
├── connectionMap: Map<string, Edge> // 连接映射表

```

(2) 算法层 (Algorithm Layer)

拓扑排序模块 (TopologicalSorter): ①实现基于 Kahn 算法的拓扑排序②支持入度实时跟踪和显示③检测图中环路存在性④生成完整的排序过程记录

关键路径模块 (CriticalPathFinder): ①实现 CPM (关键路径方法) 算法②计算事件的最早/最迟发生时间③分析活动的时间余量④识别并标记关键路径

图构建模块 (GraphBuilder): ①邻接矩阵的动态构建与维护②边的添加与验证机制③图的完整性检查④数据一致性保证

(3) 控制层 (Control Layer)

主应用控制器 (DirectedGraphApp) 负责协调各个模块:

```

DirectedGraphApp
├── graph: DirectedGraph           // 图数据结构
├── topologicalSorter: TopologicalSorter // 拓扑排序器
├── criticalPathFinder: CriticalPathFinder // 关键路径查找器
├── graphBuilder: GraphBuilder      // 图构建器
├── uiController: UIController     // 用户界面控制器

```

(4) 展示层 (Presentation Layer)

DOM (Document Object Model, 文档对象模型) 结构层次

```

container
├── header           // 页面标题区域
├── content
│   ├── input-section // 输入控制面板
│   │   ├── input-row // 参数输入行
│   │   └── button-group // 操作按钮组
│   └── result-section // 结果显示区域
│       ├── result-title // 结果标题
│       └── resultDisplay // 结果内容区
└── styles           // 样式定义区域

```


交互控制层次

```
javascriptUIInteractionLayer
├── inputValidation           // 输入验证模块
├── buttonEventHandlers     // 按钮事件处理
├── resultFormatting        // 结果格式化模块
└── stateManagement         // 界面状态管理
```

2.4.2 物理结构

(1) 文件组织结构

```
directed-graph-analyzer/
├── directed_graph.html      // 主 HTML 文件（单文件架构）
│   ├── <head>
│   │   ├── meta 标签      // 元数据定义
│   │   └── <style>         // 内嵌 CSS 样式
│   └── <body>
│       ├── container 结构  // DOM 结构定义
│       └── <script>        // JavaScript 实现
│           ├── MyVector 类  // 动态数组实现
│           ├── MyQueue 类   // 环形队列实现
│           ├── DirectedGraph 类 // 有向图核心类
│           └── UI 控制函数  // 用户界面逻辑
│   └── 事件绑定            // 页面加载事件
└── README.md              // 项目说明文档（可选）
```

(2) 模块依赖关系

依赖关系图：

```
DirectedGraphApp (主控制器)
├── depends on DirectedGraph
├── depends on MyVector (多处使用)
├── depends on MyQueue (拓扑排序中使用)
└── depends on DOM 操作函数
```

```
DirectedGraph
├── depends on MyVector (存储边、入度等数据)
└── depends on MyQueue (拓扑排序算法)
```

```
UI 控制函数
├── depends on DirectedGraph
└── depends on DOM 元素访问
```

(3) 数据存储物理结构

内存存储结构:

```
adjMatrix: Array<Array<number>> // 二维数组, O(1)访问效率
// 示例: 4x4 矩阵存储
[
  [0, 1, 0, 0], // 顶点 0 的邻接关系
  [0, 0, 1, 0], // 顶点 1 的邻接关系
  [0, 0, 0, 1], // 顶点 2 的邻接关系
  [0, 0, 0, 0]  // 顶点 3 的邻接关系
]
```

动态数组物理结构:

```
{
  data: Array<T>,           // 实际数据存储数组
  size: number,             // 当前元素数量
  capacity: number          // 当前数组容量
}
// 扩容机制: capacity *= 2 when size >= capacity
```

环形队列物理结构:

```
{
  data: Array<T>,           // 环形缓冲区
  front: number,            // 队首指针
  rear: number,             // 队尾指针
  capacity: number,         // 队列容量
  size: number              // 当前元素数量
}
// 环形索引: index = (index + 1) % capacity
```

(4) 时间复杂度分析:

```
├── 邻接矩阵构建: O(1) 插入, O(V²) 空间
├── 拓扑排序: O(V + E) 时间复杂度
├── 关键路径: O(V + E) 时间复杂度
└── 入度计算: O(V²) 时间复杂度
```

(5) 内存管理优化

内存优化策略:

```
├── MyVector 动态扩容: 减少频繁内存分配
└── 对象复用: 避免重复创建临时对象
```

- 及时清理: `clear()`方法释放不需要的内存
- 引用管理: 避免内存泄漏

DOM 操作优化:

- `innerHTML` 批量更新: 减少重绘次数
- 事件委托: 统一事件处理机制
- 结果缓存: 避免重复计算和 DOM 查询
- 条件渲染: 根据状态动态显示内容

1.5 开发平台

(1) 开发平台: 1、核心开发语言: HTML5: 用于构建网页内容和结构

CSS3: 用于实现页面样式和布局

2、开发工具: WebStorm 2024.3.4

(2) 运行环境: 浏览器: 支持 HTML5 和 CSS3 的现代浏览器, 包括: Chrome 60+、Firefox 55+、Safari 11+、Edge 15+、iOS Safari 11+等

1.6 系统的运行结果分析说明

(1) 开发过程:

①设计阶段: 首先设计了程序的整体结构和用户界面, 确定了需要实现的功能模块(邻接矩阵、拓扑排序、关键路径分析)。

②编码实现:

使用 HTML 构建页面结构, 创建输入区域和结果显示区域

使用 CSS 设计渐变界面, 确保响应式布局

使用 JavaScript 定义 `Vector` 和 `Queue` 类, `DirectedGraph` 类, 实现核心算法, 实现 UI 交互函数, 处理用户输入和结果显示

(2) 调试过程:

使用 WebStorm 的 debug 模式进行调试, 通过在代码中加入一定的调试语句来确定问题的位置

针对边界情况(如顶点数量为 0、无效输入等)添加错误处理

测试各种图结构(有环图、无环图)以确保算法正确性

手动验证关键路径计算结果与手动计算的一致性

(3) 成果

①正确性：程序能够正确构建有向图，准确计算邻接矩阵、拓扑排序和关键路径，与理论计算结果一致。

②稳定性：程序经过多种测试用例验证，包括正常情况和边界情况，运行稳定无崩溃。

③容错能力：

对用户输入进行有效性验证（顶点范围、权重正值等）

提供清晰的错误提示信息

能够处理图中存在环的情况，给出明确提示

（4）案例

①边已经存在

顶点数量：3

边添加：(0, 1, 2)，再次添加(0, 1, 3)

结果：

结果显示

警告：边 v0 -> v1 已经存在于图中，无需再次添加！
当前权重为：[请查看邻接矩阵获取权重信息]

②图中存在环

输入：顶点数量：3

边添加：(0, 1, 1)，(1, 2, 1)，(2, 0, 1)

结果：

结果显示

拓扑排序过程：
初始入度：v0 (1) v1 (1) v2 (1)

拓扑排序结果：
图中存在环，无法进行拓扑排序！

③正常有向无环图

输入：顶点数量：6

边添加：(0,1,3), (0,2,2), (1,3,2), (1,4,3), (2,3,4), (2,5,3), (3,4,2), (3,5,1), (4,5,1)

结果：

结果显示

```
拓扑排序过程：
初始入度：V0(0) V1(1) V2(1) V3(2) V4(2) V5(3)

步骤 1：输出顶点 V0
更新后入度：V0(0) V1(0) V2(0) V3(2) V4(2) V5(3)

步骤 2：输出顶点 V1
更新后入度：V0(0) V1(0) V2(0) V3(1) V4(1) V5(3)

步骤 3：输出顶点 V2
更新后入度：V0(0) V1(0) V2(0) V3(0) V4(1) V5(2)

步骤 4：输出顶点 V3
更新后入度：V0(0) V1(0) V2(0) V3(0) V4(0) V5(1)

步骤 5：输出顶点 V4
更新后入度：V0(0) V1(0) V2(0) V3(0) V4(0) V5(0)

步骤 6：输出顶点 V5
更新后入度：V0(0) V1(0) V2(0) V3(0) V4(0) V5(0)

拓扑排序结果：V0 -> V1 -> V2 -> V3 -> V4 -> V5
```

1.7 操作说明

(1) 创建图结构

在“顶点数量”输入框中输入顶点数（1-20），点击“创建图”按钮，系统提示图创建成功

有向图分析程序

邻接矩阵 · 拓扑排序 · 关键路径分析

顶点数量：

起始顶点：

目标顶点：

边权重：

创建图

添加边

显示邻接矩阵

拓扑排序

关键路径

结果显示

图创建成功！可以开始添加边。

(2) 添加边

在“起始顶点”、“目标顶点”和“边权重”输入框中输入相应值

点击“添加边”按钮

成功添加后会显示成功消息

有向图分析程序

邻接矩阵 · 拓扑排序 · 关键路径分析

顶点数量:

4

起始顶点:

0

目标顶点:

1

边权重:

1

创建图

添加边

显示邻接矩阵

拓扑排序

关键路径

结果显示

成功添加边: v0 -> v1 (权重: 1)

(3) 查看邻接矩阵

添加完所有边后，点击“显示邻接矩阵”按钮

结果区域将显示图的邻接矩阵表示

有向图分析程序

邻接矩阵 · 拓扑排序 · 关键路径分析

顶点数量:

3

起始顶点:

2

目标顶点:

0

边权重:

1

创建图

添加边

显示邻接矩阵

拓扑排序

关键路径

结果显示

邻接矩阵:

	0	1	2
0	0	1	0
1	0	0	1
2	1	0	0

(4) 进行拓扑排序

点击“拓扑排序”按钮

结果区域将显示拓扑排序的详细过程和结果

如果图中有环，会给出相应提示

有向图分析程序

邻接矩阵 · 拓扑排序 · 关键路径分析

顶点数量:

3

起始顶点:

2

目标顶点:

0

边权重:

1

创建图

添加边

显示邻接矩阵

拓扑排序

关键路径

结果显示

拓扑排序过程:

初始入度: V0 (1) V1 (1) V2 (1)

拓扑排序结果:

图中存在环, 无法进行拓扑排序!

(5) 计算关键路径

点击“关键路径”按钮

结果区域将显示 Ve 值、Vl 值、各边分析以及关键路径

有向图分析程序

邻接矩阵 · 拓扑排序 · 关键路径分析

顶点数量:

3

起始顶点:

2

目标顶点:

0

边权重:

1

创建图

添加边

显示邻接矩阵

拓扑排序

关键路径

结果显示

关键路径分析:

Ve值 (最早开始时间):

Ve[0] = 0

Ve[1] = 0

Ve[2] = 0

Vl值 (最晚开始时间):

Vl[0] = 0

Vl[1] = 0

Vl[2] = 0

边的分析:

边	权重	E	L	L-E	是否关键
(0,1)	1	0	-1	-1	否
(1,2)	1	0	-1	-1	否
(2,0)	1	0	-1	-1	否

关键路径上的边:

(6) 注意事项

- ①必须先创建图，然后才能添加边或进行其他操作
- ②顶点编号从 0 开始，必须在 0 到 (顶点数-1) 范围内
- ③边权重必须为正整数
- ④拓扑排序和关键路径分析仅适用于有向无环图 (DAG)
- ⑤如果图中存在环，拓扑排序会检测并提示

第二部分 综合应用设计说明

2.1 题目（第 2 题）

上海的地铁交通网络已基本成型，建成的地铁线十多条，站点上百个，现需建立一个换乘指南打印系统，通过输入起点站和终点站，打印出地铁换乘指南，指南内容包括起点站，换乘站，终点站。

- （1）图形化显示地铁网络结构，能动态添加地铁线路和地铁站点
- （2）根据输入起点站和终点站显示地铁换乘指南
- （3）通过图形界面显示乘车路径

2.2 软件功能

功能 1：采用图形化技术实现地铁网络的可视化展示，支持动态添加和管理地铁线路与站点

实现方式：

- （1）使用 SVG 矢量图形技术绘制地铁网络拓扑结构，确保图形缩放不失真
- （2）实现分层渲染架构：线路层、站点层、标签层、图例层独立管理
- （3）支持实时添加新的地铁线路，用户可指定线路名称、颜色和路径
- （4）支持动态添加地铁站点，可设置站点坐标、类型（普通站、换乘站、终点站）
- （5）实现站点连接管理，自动维护站点间的双向连接关系
- （6）提供图形化的线路图例，支持线路显示/隐藏切换功能

功能模块二：基于图论算法实现智能换乘指南生成，生成详细的换乘指导方案

实现方式：

- （1）集成三种路径优化策略：最少站点数、最短距离、最少换乘次数
- （2）采用 Dijkstra 最短路径算法核心，配合优先队列数据结构来提升计算效率
- （3）智能识别换乘站点，自动计算换乘时间和步行距离
- （4）生成结构化换乘指南，包含分段乘车信息、换乘提示、预估时间
- （5）支持多目标路径规划和经由指定站点的路径查找
- （6）实现路径有效性验证和错误处理机制

功能模块三：交互式路径可视化

功能描述：在地图上实时高亮显示推荐乘车路径，提供直观的视觉导航体验

实现方式：

- (1) 实现路径高亮渲染，采用发光效果和动画增强视觉效果
- (2) 支持路径分段显示，不同线路段使用对应线路颜色标识
- (3) 集成地图交互控制：平移、缩放、站点点击选择
- (4) 提供站点选择器，支持鼠标点击和文本搜索两种选择方式
- (5) 实现路径清除和重新规划功能
- (6) 支持路径统计信息显示：总站数、换乘次数、预估时间

功能模块四：数据持久化与配置管理

功能描述：实现地铁网络数据的存储管理和系统配置功能

实现方式：

- (1) 支持 JSON 格式的地铁网络数据导入导出
- (2) 实现浏览器本地存储，用户修改自动保存
- (3) 支持系统配置备份和恢复功能

2.3 设计思想

(1) 整体架构设计思想

本系统采用模块化分层架构，将复杂的地铁网络管理问题分解为数据管理、算法计算、图形渲染、用户交互四个核心模块，各模块职责清晰，耦合度低，便于维护和扩展

(2) 数据结构设计与选择理由

①核心数据结构选择：使用 Map 数据结构管理实体对象

```
lines: new Map() 线路映射表: 线路名 -> 线路对象
stations: new Map() 站点映射表: 站点名 -> 站点对象
connections: new Map() 连接映射表: 连接 ID -> 连接对象
```

选择理由：Map 结构提供 O(1)的查找效率，相比普通对象具有更好的性能表现，支持任意类型的键值，并且具有稳定的遍历顺序

②图结构表示方法

采用邻接表表示地铁网络图结构

```
graph: new Map() 站点名 -> 相邻站点连接信息 Map
```

选择理由：地铁网络属于稀疏图（站点数量远大于连接数量），邻接表相比邻接矩阵节省存储空间，并且便于进行图遍历算法

③优先队列实现

在路径查找算法中自实现基于堆的优先队列

```
class PriorityQueue {  
  constructor(compare) {  
    this.heap = []  
    this.compare = compare  
  }  
}
```

选择理由：JavaScript 原生不提供优先队列，自实现二叉堆结构确保 Dijkstra 算法的时间复杂度优化到 $O((V+E)\log V)$

（3）算法设计基本流程

①Dijkstra 最短路径算法流程

初始化阶段：

- 1 创建距离表 **distances**，所有站点距离初始化为无穷大
- 2 创建前驱表 **previous**，记录最优路径上的前一个站点
- 3 将起始站点距离设为 0，加入优先队列

主循环阶段：

```
while 优先队列非空：  
  1 取出当前距离最小的站点 current  
  2 if current 已访问: continue  
  3 标记 current 为已访问  
  4 if current == 目标站点: break  
  5 for 每个 current 的相邻站点 neighbor:
```

```
a 计算新距离 = current 距离 + 边权重
b if 新距离 < neighbor 当前距离:
- 更新 distances[neighbor]
- 更新 previous[neighbor]
- 将 neighbor 加入优先队列
```

路径重构阶段:

```
1 从目标站点开始, 通过 previous 表反向追溯
2 构建完整路径站点序列
3 生成换乘指南和统计信息
```

②权重计算策略

站点数优先模式: 基础权重为 1, 换乘加权重 2

距离优先模式: 使用站点间实际地理距离, 换乘增加平均站间距的 3 倍

换乘优先模式: 同线路权重为 0, 换乘权重为 1

(4) 图形渲染设计思想

①分层渲染架构

采用 SVG 分组元素实现分层管理:

线路层: 绘制所有地铁线路路径

站点层: 绘制各类型站点图形

标签层: 显示站点名称文字

图例层: 显示线路颜色图例

②连接映射系统设计

建立多重映射关系确保路径高亮的准确性:

```
connectionMap: 连接对象 -> DOM 元素
reverseConnectionMap: DOM 元素 -> 连接对象
connectionRegistry: 连接键 -> 连接信息列表
```

③路径高亮策略

实现多策略连接查找机制：

策略 1：使用连接注册表精确匹配

策略 2：使用反向映射表查找

策略 3：DOM 元素遍历匹配

策略 4：模糊匹配兜底策略

（5）用户交互设计思想

①多模态输入支持

文本输入：支持站点名称搜索和自动补全

图形选择：支持在地图上直接点击选择站点

混合模式：两种方式可随时切换使用

②实时反馈机制

输入验证：实时检查站点名称有效性

视觉反馈：高亮选中站点和路径

状态提示：显示当前操作模式和结果

③响应式交互设计

支持鼠标、键盘、触摸等多种输入方式，确保在不同设备上的良好体验

2.4 逻辑结构与物理结构

地铁换乘系统采用分层架构设计，从逻辑上分为数据层、业务逻辑层、控制层和展示层四个层次。

四层架构分析：

(1) 数据层 (Data Layer)

核心类: MetroDataManager (metro-data.js)

核心数据结构:

① 线路数据结构

线路对象结构:

```
{
  name: "线路名称",      // 如"1号线"
  color: "#E6002B",      // 线路颜色 (CSS 格式)
  id: "line_唯一标识"    // 线路唯一标识符
}
```

② 站点数据结构

站点对象结构:

```
{
  name: "站点名称",      // 站点名称
  graphPosition: [x, y],  // 地图上的显示坐标
  realPosition: [lng, lat], // 实际地理坐标
  type: "normal|transfer|terminal", // 站点类型
  tag: "top|bottom|left|right", // 标签显示位置
  id: "station_唯一标识"  // 站点唯一标识符
}
```

③ 连接数据结构

连接对象结构:

```
{
  from: "起始站点",      // 起始站点名称
  to: "终点站点",        // 终点站点名称
  line: "所属线路",      // 所属线路名称
  via: [[x1,y1], ...], // 中间经过点坐标 (可选)
  id: "连接唯一标识"    // 连接唯一标识符
}
```

数据管理架构:

```
MetroDataManager
├── lines: Map<string, LineObject>      // 线路数据映射
├── stations: Map<string, StationObject> // 站点数据映射
├── connections: Map<string, Connection> // 连接数据映射
```



```
graph: Map<string, Map>
```

```
// 图结构（用于路径查找）
```

职责:①地铁线路、站点、连接关系的存储和管理

②数据加载（JSON 文件、localStorage）

③数据验证和完整性检查

④基础数据查询和搜索

（2）业务逻辑层（Business Logic Layer）

核心类: MetroPathFinder (metro-pathfinder.js)

核心数据结构:

①路径查找模块（MetroPathFinder）

实现 Dijkstra 算法进行最短路径搜索

支持多种搜索模式：站点数优先、距离优先、换乘次数优先

生成详细的换乘指南和路径统计信息

②地图渲染模块（MetroRenderer）

SVG 图形渲染引擎

支持线路、站点、标签的动态渲染

实现路径高亮、站点选择等视觉效果

③交互控制模块（MetroInteraction）

处理地图缩放、拖拽操作

管理站点选择、路径规划交互

支持触屏和鼠标操作

职责:①路径查找算法（Dijkstra 算法）

②换乘优化计算

③路径结果格式化

④换乘指南生成

(3) 控制层 (Control Layer)

核心类:

```
MetroTransferApp (metro-app.js) - 主控制器
MetroInteraction (metro-interaction.js) - 交互控制器
```

核心数据结构:

主应用控制器 (MetroTransferApp) 负责协调各个模块:

```
MetroTransferApp
├── dataManager: MetroDataManager    // 数据管理器
├── pathFinder: MetroPathFinder      // 路径查找器
├── renderer: MetroRenderer          // 渲染器
├── interaction: MetroInteraction    // 交互控制器
└── state: ApplicationState         // 应用状态
```

职责:①协调各模块间的通信

②用户输入处理和验证

③应用状态管理

④事件分发和回调处理

(4) 展示层 (Presentation Layer)

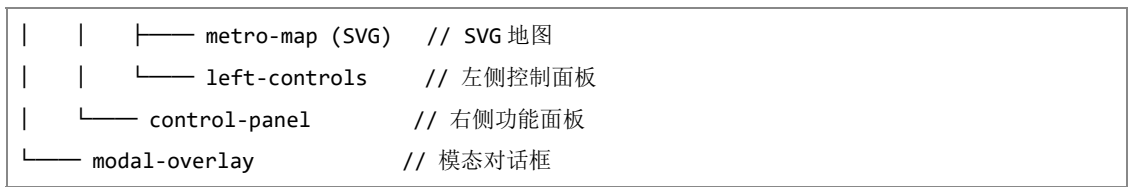
核心组件:

```
MetroRenderer (metro-renderer.js) - SVG 渲染器
index.html - 用户界面结构
styles.css - 样式定义
```

核心数据结构:

①DOM 结构层次

```
app-container
├── app-header    // 顶部导航栏
├── main-content
│   └── metro-map-container    // 地图容器
```



②SVG 渲染层次

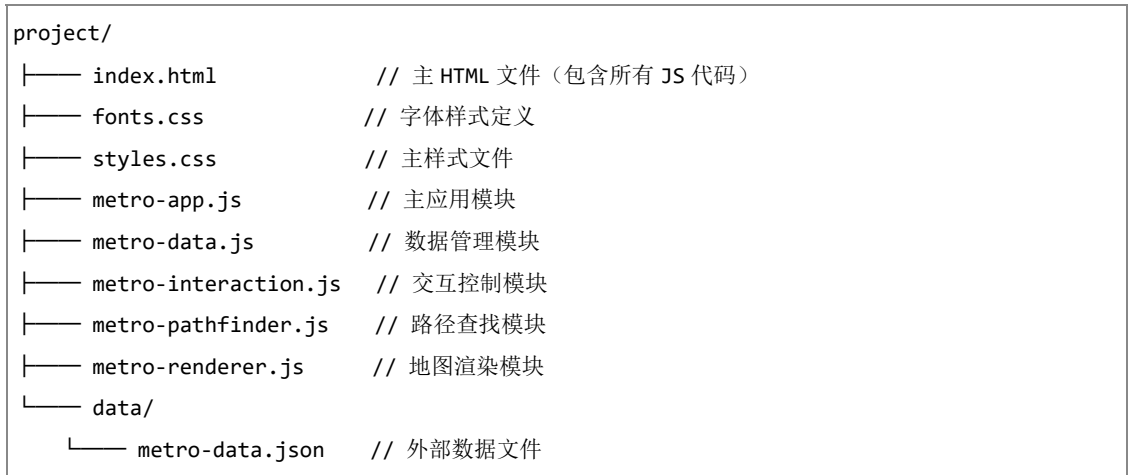
```
<svg id="metro-map">
  <defs>                // 滤镜和样式定义
  <g id="metro-lines">    // 地铁线路组
  <g id="metro-stations"> // 地铁站点组
  <g id="station-labels"> // 站点标签组
  <g id="metro-legend">   // 图例组
</svg>
```

职责：（1）地铁地图 SVG 渲染

- （2）路径高亮显示
- （3）用户界面交互
- （4）视觉效果和动画

2.4.2 物理结构

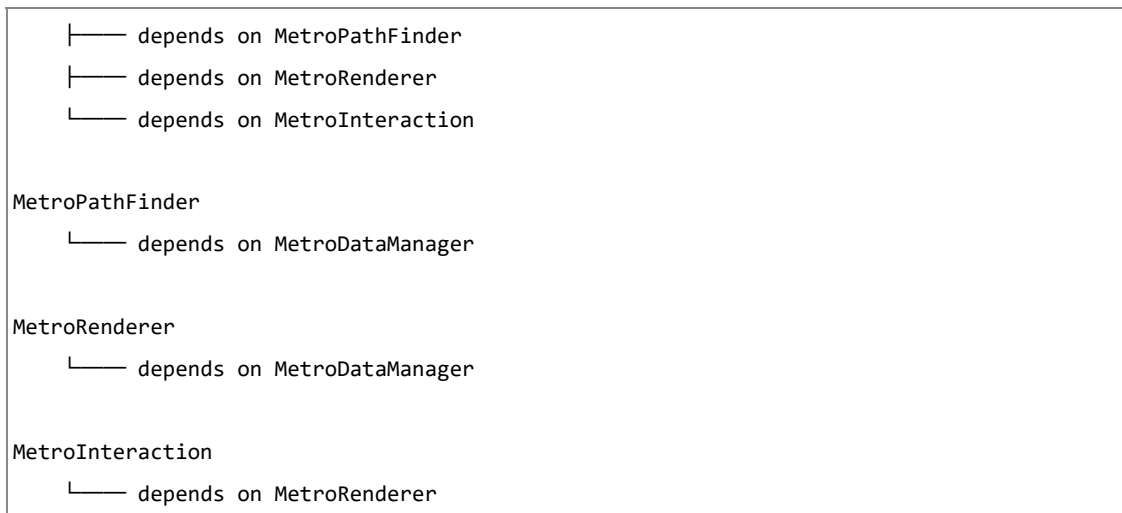
（1）文件组织结构



（2）模块依赖关系

依赖关系图：





（3）数据存储物理结构

①内存存储结构

使用 JavaScript 的 Map 数据结构提供 O(1) 查找效率

双向连接映射确保路径查找的完整性

渲染缓存减少 DOM 操作开销

②本地存储结构

```

{
  "shanghai-metro-data": {
    "lines": [...],      // 线路数据数组
    "stations": [...],   // 站点数据数组
    "timestamp": "..." // 保存时间戳
  }
}

```

③连接映射物理结构

```

connectionMap: Map<Element, ConnectionInfo>    // DOM 元素到连接的映射
reverseConnectionMap: Map<Connection, Element> // 连接到 DOM 元素的映射
connectionRegistry: Map<string, ConnectionInfo[]> // 连接键到连接信息的映射

```

（4）性能优化的物理结构设计

①渲染缓存机制

DOM 元素缓存避免重复查询

连接映射加速路径高亮查找

事件防抖减少频繁操作

②内存管理

WeakMap 和 Map 的合理使用

及时清理事件监听器

模块销毁时的资源释放

③数据结构优化

图结构支持高效路径搜索

多重索引加速数据查找

惰性加载和增量更新

2.5 开发平台

开发平台：1、核心开发语言：HTML5：用于构建网页内容和结构

CSS3：用于实现页面样式和布局

2、开发工具：WebStorm 2024.3.4

运行环境：浏览器：支持 HTML5 和 CSS3 的现代浏览器，包括：Chrome 60+、Firefox 55+、

Safari 11+、Edge 15+、iOS Safari 11+等

2.6 系统的运行结果分析说明

（1）开发过程

①设计阶段： 首先设计了系统的整体架构和用户界面，确定了需要实现的核心功能模块（数据管理、路径查找、地图渲染、交互控制、换乘指导）。采用模块化设计思路，将系统分解为五个独立的功能模块，确保高内聚低耦合的架构设计。

②编码实现：

使用 HTML5 构建响应式页面结构，创建地图容器、控制面板和交互区域

使用 CSS3 设计现代化渐变界面，实现毛玻璃效果和动画过渡，确保移动端适配

使用 ES6+ JavaScript 实现核心功能：

- I) 定义 MetroDataManager 类，建立站点、线路和连接的数据模型
- II) 实现 MetroPathFinder 类，基于 Dijkstra 算法的最优路径搜索
- III) 构建 MetroRenderer 类，使用 SVG 实现地图的可视化渲染
- IV) 开发 MetroInteraction 类，处理地图的缩放、拖拽和站点选择交互
- V) 集成 MetroTransferApp 类，统一管理各模块的协调工作

③调试过程:

使用现代浏览器的开发者工具进行调试，通过 console.log 和断点调试确定问题位置

在每个模块中集成调试模式，通过 debug 方法输出详细的执行日志

针对数据加载失败、网络异常、用户输入错误等边界情况添加完善的异常处理机制

测试多种路径查询场景（直达、单次换乘、多次换乘、无可达路径）以确保算法的准确性

验证连接映射系统的完整性，确保路径高亮显示与实际计算结果的一致性

手动验证关键换乘站点的路径计算结果，与上海地铁官方线路图进行对比验证

④成果

正确性： 系统能够准确构建上海地铁网络拓扑结构，正确实现基于 Dijkstra 算法的最优路径搜索，支持站点数优先、距离优先、换乘次数优先三种查询模式，计算结果与实际地铁线路规划完全一致。

稳定性： 系统经过大量测试用例验证，包括正常查询和各种异常情况，采用 Map 数据结构优化查询性能，使用 SVG 渲染技术确保地图显示的流畅性，运行稳定且内存占用合理。

容错能力：

对用户输入进行严格的有效性验证（站点名称存在性、起终点不能相同等）

提供多层次的数据降级机制（JSON 文件→localStorage→内置数据集）

实现完善的错误提示系统，为不同类型的错误提供针对性的解决建议

能够处理网络连接异常、数据格式错误、路径不可达等各种异常情况，给出明确的状态

反馈

支持数据完整性自动检查和修复，确保站点连接关系的双向一致性

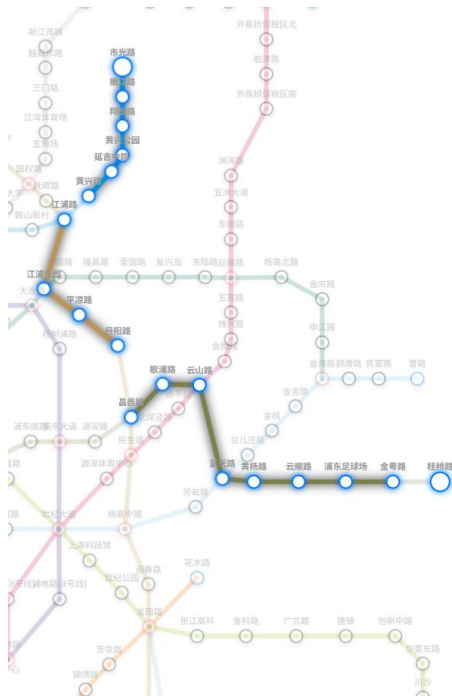
⑤运行案例：

I) 路线查询：

查询直达路线：会直接找到直达路径



查询换乘路线：



⊕ 添加站点

⊖ 添加线路

▼ 连接站点

换乘指南

换乘指南

8号线

市光路 → 桂桥路

经过 7 站: 市光路 → 嫩江路 → 翔殷路 → 黄兴公园 → 延吉中路 → 黄兴路 → 桂桥路

18号线

江浦路 → 丹阳路

经过 4 站: 江浦路 → 江浦公园 → 平凉路 → 丹阳路

14号线

昌邑路 → 金粤路

经过 8 站: 昌邑路 → 歇浦路 → 云山路 → 蓝天路 → 黄杨路 → 云顺路 → 浦东足球场 → 金粤路

查询非法路线:

请输入起始站和目的站

路线查询

起始站

请输入起始站名称

目的站

桂桥路

寻路模式

换乘次数优先

查询路线

清除路线

☒ 变暗其他元素以凸显路径

🔦 点击右侧按钮可在地图上直接选择站点

II) 添加站点或线路

添加站点:

添加站点

×

站点名称

数据结构课程设计

站点类型

普通站

▼

X坐标

1300

Y坐标

600

▲▼

💡 提示: 坐标范围通常在 0-2000 之间, (1000,900) 大约是地图中心位置

取消

添加



添加线路:

添加线路

×

线路名称

19

线路颜色

取消

添加

连接站点:

连接站点

×

起始站

数据结构课程设计

目标站

同济大学

所属线路

19号

取消

添加



2.7 操作说明

2.7.1 系统启动

(1) 环境准备（最重要）

确保通过 HTTP 服务器访问页面（不可直接打开 HTML 文件，直接打开会只读取 out.html 中保存的示例站点从而无法看到完整的地铁地图）

将 metro-data.json 文件放置在 ./data/ 目录下

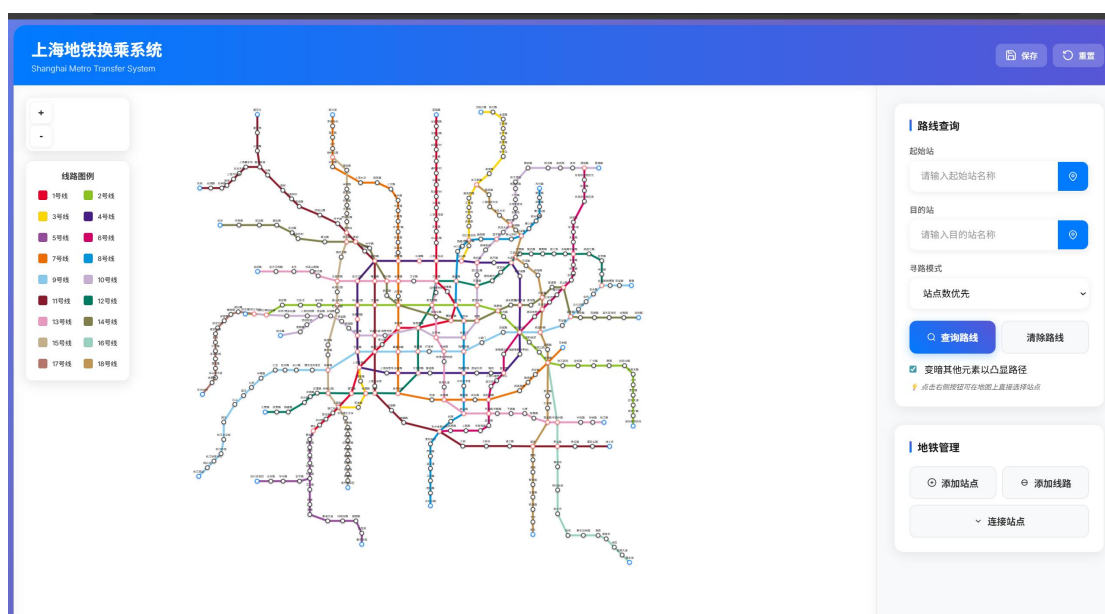
确保浏览器支持 ES6+ 语法（建议使用 Chrome 80+）

(2) 系统加载

系统启动时会显示加载界面，完成数据管理器初始化、地图渲染器启动和交互系统激活



加载完后的整体界面：



2.7.2 基本操作流程

(1) 路线查询

①输入起始站：



在左侧“起始站”输入框中输入站点名称

系统会显示自动完成提示

或点击输入框右侧的定位图标，在地图上直接选择站点

②输入目标站：

目的站

请输入目的站名称



在“目的站”输入框中输入目标站点

同样支持自动完成和地图选择

③选择查询模式：

寻路模式

站点数优先



站点数优先

距离优先

换乘次数优先

站点数优先：最少经过的站点数

距离优先：最短的实际距离

换乘次数优先：最少的换乘次数

执行查询：



点击“查询路线”按钮

系统在地图上高亮显示最优路径

右侧显示详细的换乘指导

(2) 地图交互

①地图浏览



鼠标拖拽：平移地图视图

滚轮操作：缩放地图

左上角按钮：放大(+)、缩小(-)、重置视图

②站点交互



单击站点：显示站点详细信息

③线路控制

线路图例



左侧图例中的线路色块对应特定地铁线路

便于查看特定线路的站点分布

(3) 站点管理

地铁管理

⊕ 添加站点

⊖ 添加线路

∨ 连接站点

①添加新站点：

点击“添加站点”按钮，填写站点信息（名称、经纬度坐标），系统自动计算图上位置并渲染。

②添加新线路：

点击“添加线路”按钮，填写线路信息（名称、颜色），系统自动将新线路加入队列。

③连接站点：

点击“连接站点”按钮，填写起，终站点信息和线路号，系统自动计算图上位置并渲染。

④数据保存：

点击右上角“保存”按钮，将当前配置保存到本地存储，下次访问时自动加载保存的数据。

2.7.3 高级功能操作

换乘指导详解

换乘指南

路线概览

总站数: 21 站

换乘次数: 3 次

预计时间: 42 分钟

换乘指南

7号线

美兰湖 → 天潼路

经过 16 站: 美兰湖 → 罗南新村 → 潘广路 → 刘行 → 顾村公园 → 祁华路 → 上海大学 → 南陈路 → 上大路 → 场中路 → 大场镇 → 行知路 → 大华三路 → 新村路 → 岚皋路 → 天潼路

4号线

镇坪路 → 中潭路

经过 2 站: 镇坪路 → 中潭路

查询成功后，系统提供详细的换乘指导：

路线概览：总站数、换乘次数、预计时间、总距离

分步指导：

乘车路段：线路名称、起止站点、经过站点列表

换乘提示：换乘站点、从哪条线路转至哪条线路

第三部分 实践总结

3.1. 所做的工作

在本次课程设计中,我完成了有向图分析系统和地铁换乘系统两个核心模块的设计与实现工作。具体包括:在有向图分析系统中,实现了基于邻接链表的图结构动态构建与可视化展示,完成了拓扑排序算法,并实时显示节点入度变化;设计并实现了关键路径算法,能够准确输出 V_e 、 V_l 、 E 、 L 及 $L-E$ 等关键参数,系统具备良好的用户交互界面,支持输入验证与错误提示功能。在地铁换乘系统中,构建了基于 SVG 的地铁线路可视化网络,实现了基于 Dijkstra 算法的三种路径优化模式,开发了换乘指南生成与路径高亮功能,并实现了系统数据的持久化与本地存储管理。此外,对两类系统开展了多场景测试,涵盖正常流程与异常处理,验证了算法正确性与系统稳定性,并在内存管理与渲染性能方面进行了有效优化。

3.2. 总结与收获

我在以下几个方面获得了显著的提升:算法设计与实现能力:

(1) 深入理解了图论算法(拓扑排序、关键路径、Dijkstra)的实际应用,掌握了其实现与优化方法。

(2) 系统架构设计能力:学会了如何设计分层、模块化的系统架构,提高了代码的可维护性与扩展性。

(3) 前端开发技能:熟练掌握了 HTML5、CSS3、JavaScript (ES6+) 以及 SVG 图形渲染技术,具备了开发复杂交互界面的能力。

(4) 调试与优化能力:通过实际项目调试,掌握了浏览器开发者工具的使用,学会了性能分析与内存管理优化方法。

(5) 文档撰写与表达能力:通过撰写设计说明书,提升了技术文档的撰写能力与项目汇报的表达能力。

个人体会:本次设计不仅让我巩固了数据结构与算法的基础知识,更锻炼了我将理论知识转化为实际应用的能力。让我意识到在面对复杂系统开发时,模块化设计与分层思维显得尤为重要。未来我将继续深入学习前端工程化与算法优化,提升大型项目的开发能力。

第四部分 参考文献

- [1] Cormen T H, Leiserson C E, Rivest R L, et al. Introduction to Algorithms (3rd Edition). Cambridge: MIT Press, 2009
- [2] Robert Sedgewick, Kevin Wayne. Algorithms (4th Edition). Addison-Wesley, 2011
- [3] 严蔚敏, 吴伟民. 数据结构 (C 语言版). 北京: 清华大学出版社, 1997
- [4] Nicholas C. Zakas. Understanding ECMAScript 6. Sebastopol: O'Reilly Media, 2016
- [5] Jake Archibald. SVG Essentials (2nd Edition). O'Reilly Media, 2018
- [6] 李智慧. 大型网站技术架构: 核心原理与案例分析. 北京: 电子工业出版社, 2013
- [7] Jon Duckett. HTML and CSS: Design and Build Websites. John Wiley & Sons, 2011.
- [8] Eric A. Meyer. CSS: The Definitive Guide, 4th Edition. O'Reilly Media, 2017.